Polytech Dijon

Mécatronique

Travaux Pratique

Compte rendu

Réalisé par :
ADIDO Caurel
BAKAI Mamalineso
CLOIX Mattys

Supervisé par :
ABADI Amine

24 septembre 2026

Table des matières

1 TP1 :Présentation et programmation de la carte Arduino 2
1.1 Matériel 3
1.2 Le logiciel Arduino 4
1.3 Manipulation 4
1.3.1 Manipulation 1 : Une LED qui clignote 4
1.3.2 Manipulation 2 : Trois LED qui clignote 5

2 TP2 :Utilisation du CAN de l'Arduino 11
2.1 Convertisseur Analogique numérique 12
2.2 Mesurer une température avec un capteur LM35 et une carte Arduino 12
2.3 Mesurer la luminosité avec une photorésistance et une carte Arduino 14
2.4 Variation de la luminosité d'une LED avec un potentiomètre et une carte Arduino 17

Chapitre 1

TP1 :Présentation et programmation de la carte Arduino

1.1 Matériel

Le projet Arduino a été conçu par une équipe de six développeurs pour offrir une plateforme de création de systèmes électroniques accessible à tous, des débutants aux professionnels.

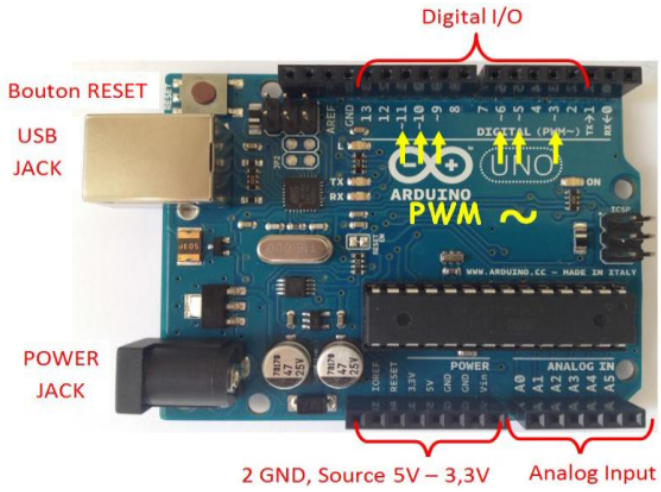


FIGURE 1.1 – Arduino Uno

La carte Arduino UNO est un circuit imprimé architecturé autour du microcontrôleur ATmega328. Elle est équipée d'une interface USB permettant une communication directe avec un ordinateur.

Caractéristiques techniques principales :

- **Tensions :** Fonctionnement sous 5V, avec une tension d'alimentation recommandée de 7 à 12V.
- **Performances :** Horloge cadencée à 16 MHz et mémoire Flash de 32 Ko.
- **Courant :** Jusqu'à 40 mA par broche d'entrée/sortie (50 mA pour la broche 3.3V).
- **Brochage :**
  - *Entrées analogiques :* A0 à A5
  - *Entrées/Sorties numériques :* 2 à 19
  - *Sorties PWM (analogiques simulées) :* 3, 5, 6, 9, 10, 11

1.2 Le logiciel Arduino

La programmation de la carte Arduino nécessite un logiciel pour écrire le code puis le transférer. Nous utiliserons pour cela Arduino IDE, une solution gratuite et open source.

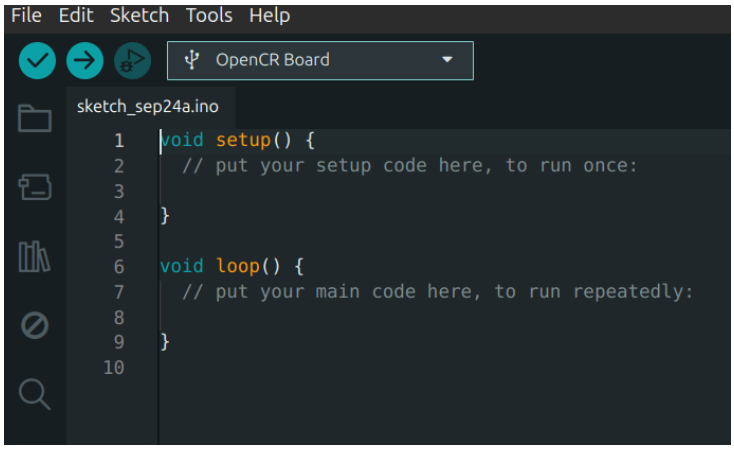


FIGURE 1.2 – Interface Arduino IDE

1.3 Manipulation

1.3.1 Manipulation 1 : Une LED qui clignote

Le but de cette première manipulation est de nous familiariser avec la carte Arduino Uno en faisant simplement clignoter une LED.

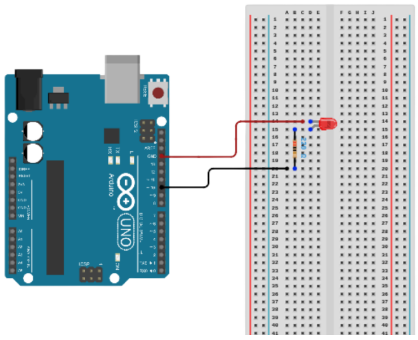


FIGURE 1.3 – Schéma Manipulation 1

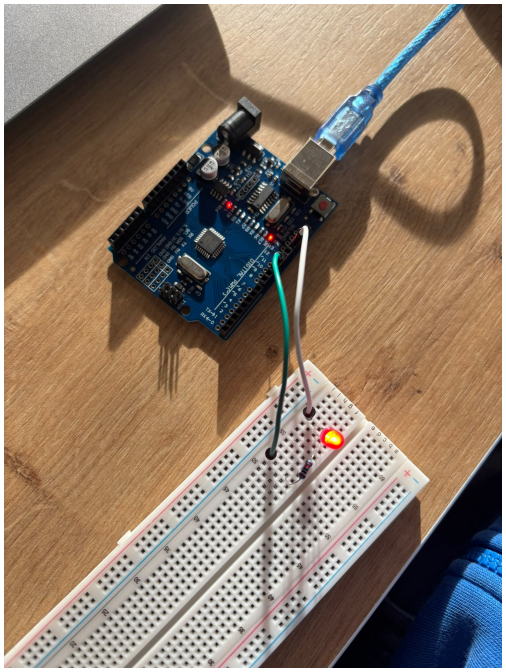


FIGURE 1.4 – Photo Manipulation 1

Voici le code qui nous est donné pour tester notre LED

```
1  void setup() {
2      pinMode(10, OUTPUT); // ici nous definissons notre
           pin 10 comme une sortie
3  }
4  void loop() {
5      digitalWrite(10, HIGH); // ici on allume la LED
6      delay(500); // on attends 500 millisecondes
7      digitalWrite(10, LOW);
8      delay(500); // on attends 500 millisecondes
9  }
```

Observation : Nous remarquons bien que la LED clignote.

1.3.2 Manipulation 2 : Trois LED qui clignote

Dans cette partie du TP, nous devons apprendre à gérer trois LED en même temps avec notre Arduino.

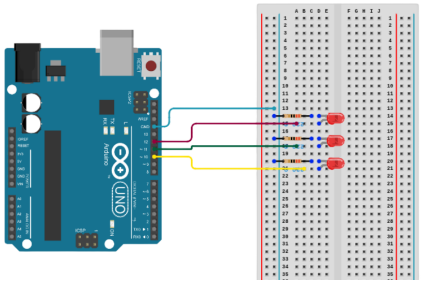


FIGURE 1.5 – Schéma manipulation 2

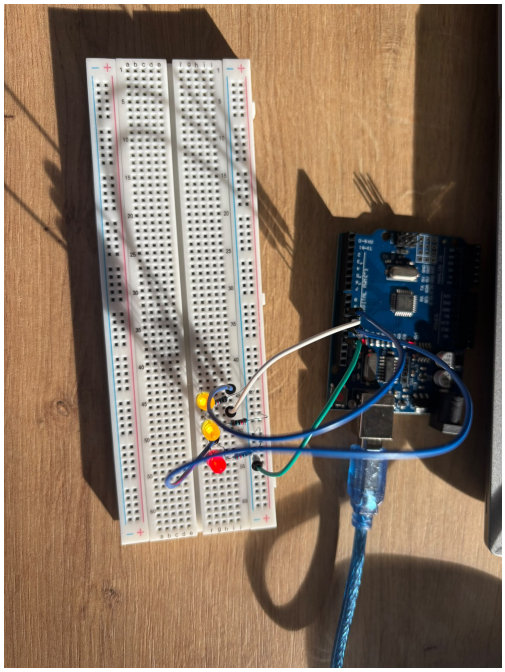


FIGURE 1.6 – Photo Manipulation 2

Voici notre code pour faire code pour faire clignoter les trois LED

```
1  void setup() {
2      pinMode(10, OUTPUT); // ici nous definissons notre
           pin 10 comme une sortie
3      pinMode(11, OUTPUT); // ici nous definissons notre
           pin 11 comme une sortie
4      pinMode(12, OUTPUT); // ici nous definissons notre
           pin 12 comme une sortie
5  }
6  void loop() {
7      digitalWrite(10, HIGH); // ici on allume la LED
8      digitalWrite(11, HIGH); // ici on allume la LED
9      digitalWrite(12, HIGH); // ici on allume la LED
10
11     delay(500); // on attends 500 millisecondes
12
13     digitalWrite(10, LOW);
14     digitalWrite(11, LOW);
15     digitalWrite(12, LOW);
16
17     delay(500); // on attends 500 millisecondes
18 }
```

Observation : Nous remarquons que les trois LED s'allume simultanément.

Ensuite nous devons faire en sorte que les trois Led s'allume avec alternance. Voici le code pour faire cela.

```
    void loop() {
        digitalWrite(10, HIGH); // ici on allume la LED
        delay(500); // on attends 500 millisecondes
        digitalWrite(10, LOW); // ici on allume la LED

        delay(500);

        digitalWrite(11, HIGH); // ici on allume la LED
        delay(500); // on attends 500 millisecondes
        digitalWrite(11, LOW); // ici on allume la LED

        delay(500);

        digitalWrite(12, HIGH); // ici on allume la LED
        delay(500); // on attends 500 millisecondes
        digitalWrite(12, LOW); // ici on allume la LED

        delay(500);
    }
```

Observations : Nous remarquons bien une alternance dans l'allumage des LED.
Pour la prochaine questions nous devons réaliser un feu tricolore avec nos trois LED de couleur. Voici alors notre code pour réaliser cela.

```
    #define LED_rouge  12 // ici nous renommons nous leds
       pour mieux les distinguer
    #define LED_vert   11
    #define LED_orange 10
    void setup() {
        pinMode(LED_rouge, OUTPUT);
        pinMode(LED_vert, OUTPUT);
        pinMode(LED_orange, OUTPUT);
    }
    void loop() {

        digitalWrite(LED_orange, HIGH);
        digitalWrite(LED_vert, LOW);
        digitalWrite(LED_rouge, LOW);

        delay(1000);

        digitalWrite(LED_orange, LOW);
        digitalWrite(LED_vert, LOW);
        digitalWrite(LED_rouge, HIGH);

```

```
        delay(4000);

        digitalWrite(LED_orange, LOW);
        digitalWrite(LED_vert, HIGH);
        digitalWrite(LED_rouge, LOW);

        delay(4000);
    }
```

Observations : Nous remarquons bien le fait que les LED s'allume a l'image d'un feux tricolore.
L'exercice suivant consiste à répéter 15 fois le clignotement d'une seule LED. Pour éviter d'alourdir le programme, nous allons conserver notre structure actuelle et introduire une boucle for.

```
    void setup() {
        Serial.begin(9600);

        pinMode(10, OUTPUT);

        for(int i=0 ; i <15 ; i++){ // cette boucle for nous
            permet d'executer le code 15 fois
            Serial.println(i);
            digitalWrite(10,HIGH);
            delay(500);
            digitalWrite(10,LOW);
            delay(500);
        }
    }
    void loop() {
        // ici on ne mets rien car on souhaite que le code
            sexecute que 15 fois
    }
```

Observations : Nous vérifions bien que la LED s'allume et s'éteint 15 fois. Nous avons désormais un code a tester.

```
    int ledPin = 10; //On renomme la broche 10 en "ledPin"
    int timer = 100; //On definit une duree de 0,1 seconde
       pour la variable timer

    void setup() { pinMode(ledPin, OUTPUT); }

    void loop() {
        // LED a 0%.
        analogWrite(ledPin, 0);
        delay(timer);
        // LED a 19.6%.
        analogWrite(ledPin, 50);
```

```
        delay(timer);
        // LED a 39.2%.
        analogWrite(ledPin, 100);
        delay(timer);
        // LED a 58.8%.
        analogWrite(ledPin, 150);
        delay(timer);
        // LED a 78.4%.
        analogWrite(ledPin, 200);
        delay(timer);
        // LED a 100%.
        analogWrite(ledPin, 255);
        delay(timer); }
```

Observations : La LED augmente en luminosité jusqu'à s'éteindre.
Maintenant que nous avons compris ce précedent code nous devons jouer avec la luminosité des Trois différente LED.

```
int ledPin = 10; //On renomme la broche 10 en "ledPin"
int timer = 100; //On definit une duree de 0,1 seconde
    pour la variable timer

void setup() {
    pinMode(ledPin, OUTPUT);
}

void loop() {
    analogWrite(10, (255*10)/100); // ici on definit la
        luminosite de la led_10 a 10% de 255
    analogWrite(11, (255*40)/100); // ici on definit la
        luminosite de la led_11 a 40% de 255
    analogWrite(12, (255*80)/100); // ici on definit la
        luminosite de la led_12 a 80% de 255

    delay(timer);

    analogWrite(10, (255*0)/100); // ici on definit la
        luminosite de la led_10 a 0% de 255
    analogWrite(11, (255*0)/100); // ici on definit la
        luminosite de la led_11 a 0% de 255
    analogWrite(12, (255*0)/100); // ici on definit la
        luminosite de la led_12 a 0% de 255

    delay(timer);
}
```

Observations : Les trois LED clignote mais avec une intensité lumineuse différente.
Pour finir avec ce TP nous allons réaliser un chenillard avec nos trois LED.

```
1   int timer = 500; //On definit une duree de 0,5 seconde
        pour la variable timer
2   void setup() {
3       pinMode(10, OUTPUT);
4       pinMode(11, OUTPUT);
5       pinMode(12, OUTPUT);
6   }
7   void loop() {
8       for (int i = 10 ; i < 13 ; i++) {
9           digitalWrite(i, HIGH);
10          delay(timer);
11          digitalWrite(i, LOW);
12      }
13  }
```

Observations : Nous observons bien l'allumage des LED de façons successive.

Chapitre 2

TP2 :Utilisation du CAN de l'Arduino

2.1 Convertisseur Analogique numérique

La carte Arduino Uno intègre un unique convertisseur analogique/numérique (CAN) de 10 bits, partagé entre ses 6 entrées (A0 à A5). Il convertit le signal en environ 100 µs pour délivrer une valeur numérique allant de 0 à 1023.

2.2 Mesurer une température avec un capteur LM35 et une carte Arduino

Le LM35 est un capteur de température électronique analogique de précision. Il se présente sous la forme d'un composant à trois broches :

— **VCC** : Alimentation (reliée au 5V)

— **Analog Voltage Out** : Sortie de la tension (reliée à une entrée analogique)

— **GND** : Masse

Principe de mesure : D'après la fiche technique du composant, la tension de sortie est proportionnelle à la température avec une sensibilité de 10 mV/°C. Autrement dit, une tension de 1 V correspond à 100°C.

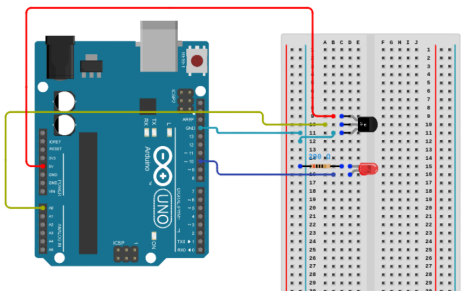


FIGURE 2.1 – Schéma de l'utilisation du LM35

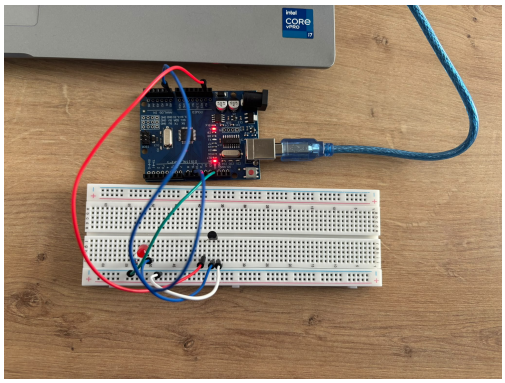


FIGURE 2.2 – Photo du montage du LM35

Pour commencer nous allons programmer la carte pour uniquement récupérer les informations du LM35.

```arduino
const int lm35Pin = A0; // Broche du capteur LM35

void setup() {
    Serial.begin(9600); // Initialisation de la communication serie
}
void loop() {
    int val = analogRead(lm35Pin); // Lecture de la valeur analogique (0 a 1023)

    // Conversion de la valeur lue en tension (en mV), puis en temperature (C)

    float voltage = val * (5.0 / 1023.0);
    float temperature = voltage * 100.0; // 10 mV par C donc T = tension * 100

    Serial.print("Temperature : ");
    Serial.print(temperature);
    Serial.println(" C");

    delay(1000); // Pause d'une seconde entre chaque mesure
}
```

Observations : Nous utilisons le moniteur série de l'Arduino pour relever la température mesurée par le capteur. La valeur obtenue étant d'environ 22 °C, nous pouvons en déduire que le montage et la programmation ont été réalisés avec succès.

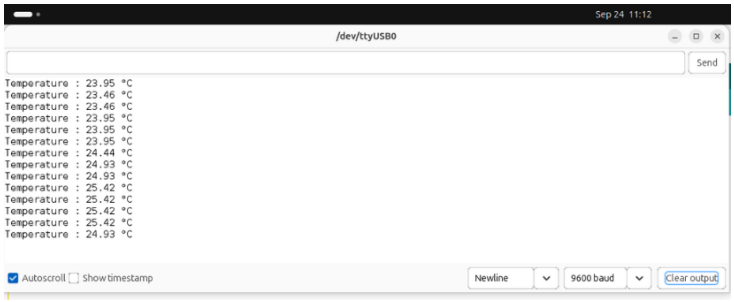


FIGURE 2.3 – Monitor série du capteur LM35

Nous faisons évoluer le programme pour utiliser la LED comme indicateur d'alerte. Si la température mesurée par le LM35 est supérieure à 35 °C, nous commandons l'allumage de la LED.

```arduino
const int lm35Pin = A0; // Broche du capteur LM35
void setup() {
```

```
    Serial.begin(9600); // Initialisation de la communication serie
    pinMode(10, OUTPUT);
}
void loop() {
    int val = analogRead(lm35Pin); // Lecture de la valeur analogique (0 a 1023)

    // Conversion de la valeur lue en tension (en mV), puis en temperature (C)

    float voltage = val * (5.0 / 1023.0);
    float temperature = voltage * 100.0; // 10 mV par C donc T = tension * 100

    Serial.print("Temperature : ");
    Serial.print(temperature);
    Serial.println(" C");

    delay(1000); // Pause d'une seconde entre chaque mesure

    if (temperature > 29) {
        digitalWrite(10,HIGH); // on allume la LED
    }else {
        digitalWrite(10,LOW); // on eteint la LED
    }
}
```

Observations : Lorsque que nous plaçons notre doigts sur le LM35 alors la LED s'allume car nous dépassons le seuil de température de notre programme.

2.3 Mesurer la luminosité avec une photorésistance et une carte Arduino

Une **photorésistance** est un composant électronique dont la valeur (en ohms) varie en fonction de la lumière ambiante : **plus elle est éclairée, plus sa résistance diminue**.

Caractéristiques classiques

- **Dans l'obscurité :** Résistance très élevée (environ 1 MΩ).
- **En pleine lumière :** Résistance plus faible (environ 12 kΩ).

Applications principales Ce comportement permet de mesurer la luminosité ambiante pour diverses utilisations :

— Détection de l'état jour/nuit.

— Ajustement automatique de l'éclairage.

— Systèmes de suivi de lumière (panneaux solaires, robotique).

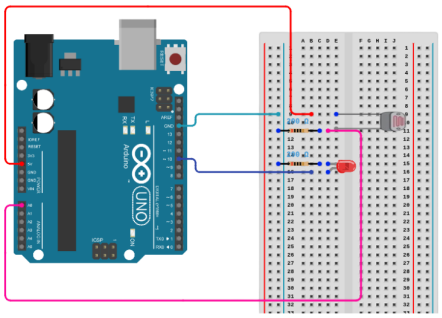


FIGURE 2.4 – Schéma du montage avec photorésistance

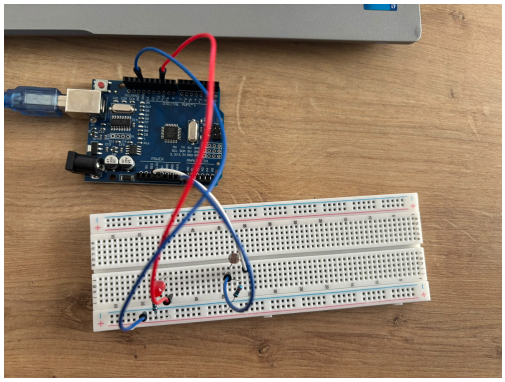


FIGURE 2.5 – Photo montage avec la photorésistance

Ensuite, nous allons faire un programme permettant de gérer l'allumage d'une LED en fonction de la luminosité reçue par la photorésistance.

```
1   // Initialisation des constantes :
2   const int analogInPin = A0; // Numero de la broche a
        laquelle est connectee la photoresistance
3   const int analogOutPin = 10; // Numero de la broche a
        laquelle est connectee la LED
4
5   int sensorValue = 0; // Valeur lue sur la photoresistance
6   int outputValue = 0; // Valeur envoyee a la LED
7   void setup() {
8       // Initialise la communication avec l'ordinateur
9       Serial.begin(9600);
10      // Indique que la broche analogOutPin est une sortie
            :
11      pinMode(analogOutPin, OUTPUT);
12      // Indique que la broche analogInPin est une entree :
13      pinMode(analogInPin, INPUT);
14  }
15  void loop() {
16      // lit la valeur de la photoresistance et
17      // stocke le resultat dans sensorValue :
18      sensorValue = analogRead(analogInPin);
```

```
19          // change sensorValue vers une intervalle de 0 a 255
20          // et stocke le resultat dans outputValue :
21          outputValue = map(sensorValue, 0, 1023, 0, 255);
22          // envoie de cette nouvelle valeur sur la LED
23          analogWrite(analogOutPin, outputValue);
24          Serial.print("sensor = " );
25          Serial.print(sensorValue);
26          Serial.print("\t output = ");
27          Serial.println(outputValue);
28      }
```

Observations : Lorsque nous mettons notre doigt sur la photorésistance pour baiser la luminosité alors la LED s'allume. Nous allons rajouter une autre LED qui s'allume avec un autre seuil.

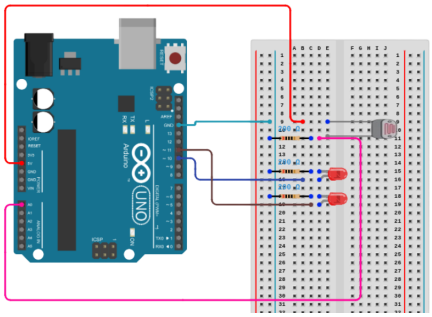


FIGURE 2.6 – Schéma du montage avec photorésistance et deux LED

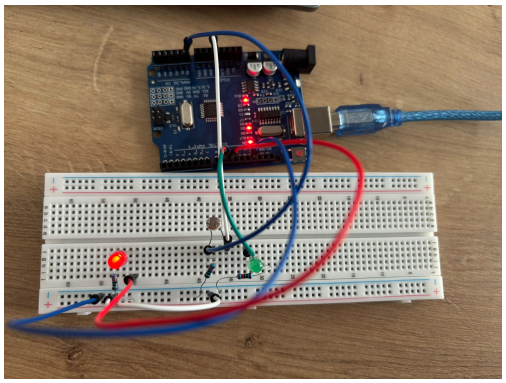


FIGURE 2.7 – Photo montage avec la photorésistance et deux LED

```
1   void loop() {
2       sensorValue = analogRead(analogInPin);
3
4       outputValue = map(sensorValue, 0, 1023, 0, 255);
5
6       analogWrite(analogOutPin, outputValue);
7
8       Serial.print("sensor = " );
9       Serial.print(sensorValue);
10      Serial.print("\t output = ");
11      Serial.println(outputValue);
12      if(sensorValue < ((1023*1)/100) ){
13          digitalWrite(11, HIGH);
14      }else{
15          digitalWrite(11, HIGH);
16      }
17  }
```

Observations : Les deux LED s'allume bien lorsque en fonction du seuil de luminosité atteint.

2.4 Variation de la luminosité d'une LED avec un potentiomètre et une carte Arduino

Pour terminer cette manipulation, nous intégrons un potentiomètre au lieu d'un capteur automatique. Cela nous donne la possibilité de contrôler manuellement le circuit en modifiant la valeur de sa résistance.

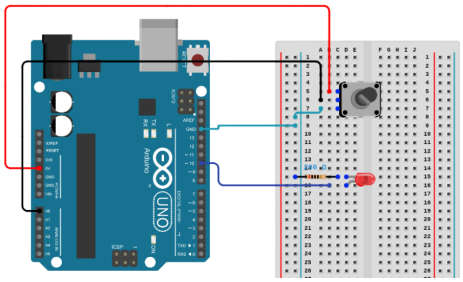


FIGURE 2.8 – Schéma du montage avec le potentiomètre

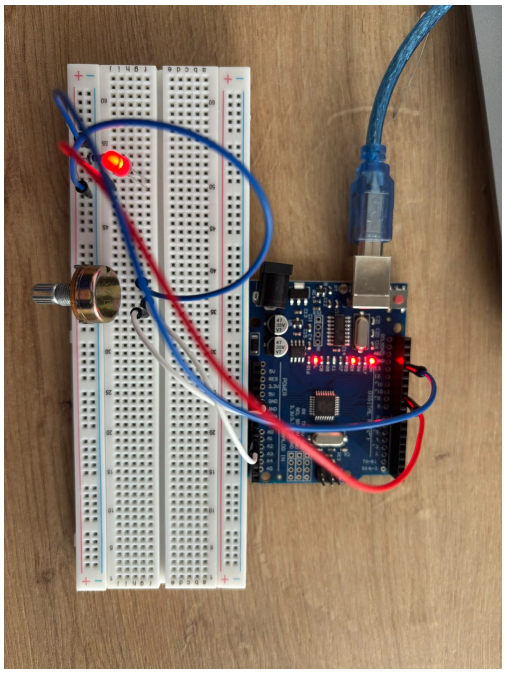


FIGURE 2.9 – Photo montage avec le potentiomètre

Nous modifions alors un peu le code de la photorésistance pour utiliser le potentiomètre.

```
1   void loop() {
2
3       sensorValue = analogRead(analogInPin);
4
5       outputValue = map(sensorValue, 0, 1023, 0, 255);
6
7       analogWrite(analogOutPin, outputValue);
8
```

```
9           Serial.print("sensor = " );
10          Serial.print(sensorValue);
11          Serial.print("\t output = ");
12          Serial.println(outputValue);
13      }
```

Observations : La LED s'allume bien lorsque que nous tournons le potentiomètre.
Nous ajoutons ensuite une autre LED qui devras s'allumer quand le potentiomètre est au minimum.

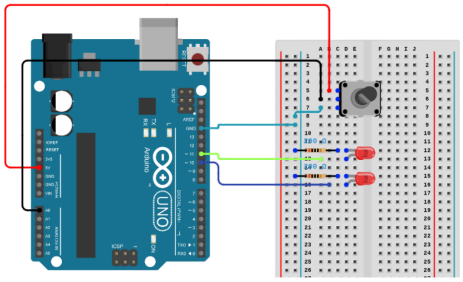


FIGURE 2.10 – Schéma du montage du potentiométre avec 2 LED

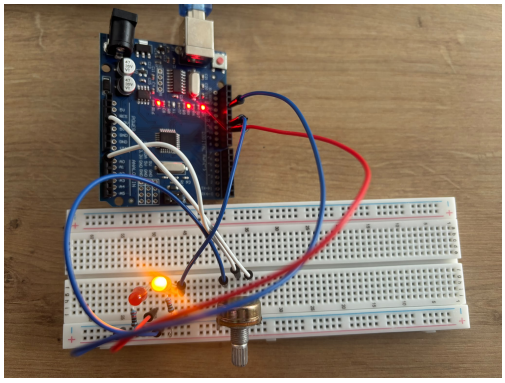


FIGURE 2.11 – Photo montage du potentiométre avec 2 LED

```
1       void loop() {
2
3           sensorValue = analogRead(analogInPin);
4
5           outputValue = map(sensorValue, 0, 1023, 0, 255);
6           outputValue2 = map(sensorValue, 0, 1023, 255, 0); //
                on inverse le 255
7
8           analogWrite(analogOutPin, outputValue);
9           analogWrite(11, outputValue2);
10
11          Serial.print("sensor = " );
12          Serial.print(sensorValue);
13          Serial.print("\t output = ");
14          Serial.println(outputValue);
15
16      }
```

Observations : Tourner le potentiomètre dans un sens allume une première LED, le tourner dans l'autre sens allume la seconde.

Pour finir avec ce TP nous devons afficher sur un écran LCD la température que nous mesurons a l'aide du LM35 utilisé dans la première partie du TP. Pour ce faire nous utilisons un code qui a été fournie par une bibliothèque trouvé sur Google.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
const int lm35Pin = A0; // Broche du capteur LM35
// Initialisation du LCD I2C : Adresse 0x27, 16 colonnes, 2 lignes
LiquidCrystal_I2C lcd(0x27, 16, 2);

void setup() {
    // Initialisation du LCD
    lcd.init();
    lcd.backlight(); // Allume le retroeclairage
    lcd.clear();
}
void loop() {
    int val = analogRead(lm35Pin); // Lecture du capteur

    // Conversion en temperature (C)
    float voltage = val * (5.0 / 1023.0);
    float temperature = voltage * 100.0;

    // Affichage sur l'ecran LCD
    lcd.setCursor(0, 0); // Ligne 1
    lcd.print("Temp: ");
    lcd.print(temperature, 1); // 1 chiffre apres la virgule
    lcd.print(" C   "); // Espaces pour eviter les bugs d'affichage

    delay(1000); // Pause d'une seconde avant la prochaine mesure
}
```

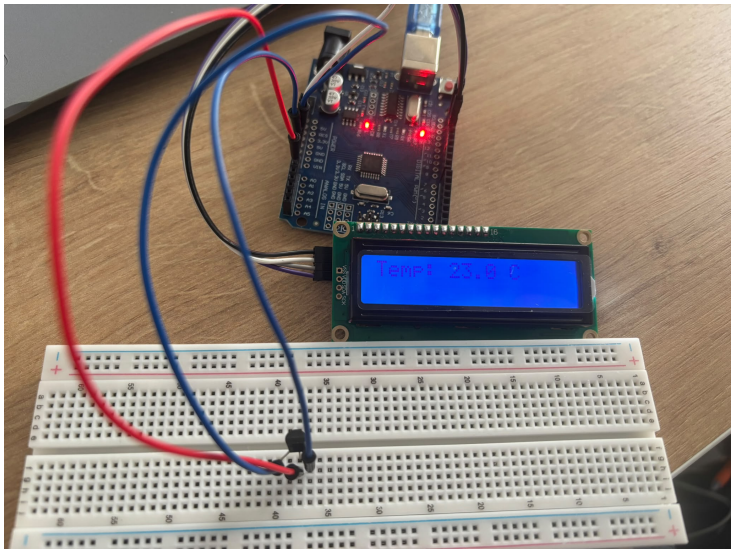


FIGURE 2.12 – Montage de l'affichage sur l'écran LCD